potential for a race condition to define first. On many pre-C++11 compilers this race condition is problematic in practice, because multiple threads may believe they're first and try to initialize the variable, or threads may try to use it after initialization has started on another thread but before it's finished. In C++11 this problem is solved: the initialization is defined to happen on exactly one thread, and no other threads will proceed until that initialization is complete, so the race condition is just over which thread gets to do the initialization rather than anything more problematic. This can be used as an alternative to `std::call_once` for those cases where a single global instance is required:

```
class my_class;
my_class& get_my_class_instance()
{
    static my_class instance;
    return instance;
}
```

❶ Initialization guaranteed to be thread-safe

Multiple threads can then call `get_my_class_instance()` safely ❶, without having to worry about race conditions on the initialization.

Protecting data only for initialization is a special case of a more general scenario: that of a rarely updated data structure. For most of the time, such a data structure is read-only and can therefore be merrily read by multiple threads concurrently, but on occasion the data structure may need updating. What's needed here is a protection mechanism that acknowledges this fact.

3.3.2. Protecting rarely updated data structures

Consider a table used to store a cache of DNS entries for resolving domain names to their corresponding IP addresses. Typically, a given DNS entry will remain unchanged for a long period of time—in many cases DNS entries remain unchanged for years. Although new entries may be added to the table from time to time as users access different websites, this data will therefore remain largely unchanged throughout its life. It's important that the validity of the cached entries be checked periodically, but this still requires an update only if the details have actually changed.